

Practice Question Set

1. What is a structure in C, and how does it differ from a standard array?
2. Explain the purpose of the struct keyword and what a tag name represents.
3. Are memory locations assigned immediately when a structure template is defined? Explain.
4. Write out the template definition for a structure named student containing a character array name, an integer roll number, and a float marks value.
5. What are the two distinct methods available to instantiate structure variables?
6. What is a Sequential Initializer List, and what happens if you provide fewer values than the total number of structure members?
7. Which operator is used to access structure members through a direct instance variable?
8. Write a code snippet showing how to use scanf() to read a structure variable with a custom forward slash (/) delimiter scheme.
9. How can you smoothly copy all properties from one structure variable instance to another variable instance of the exact same structure type?

10. Can you pass complete structure instances as parameter variables to a function, or return entire structure blocks from a function?

Important Interview/Viva Questions

11. Explain the purpose of the arrow operator (->) and write its completely identical bracketed indirection equivalent notation.

12. What is the role of the typedef keyword when defining structures? Provide an example of how it cleans up object declarations.

13. Explain what Structure Padding is and why processors read data in fixed alignments rather than 1 byte at a time.

14. Given the following structure, calculate its total evaluated size in bytes on a standard 32-bit CPU boundary, accounting for padding:

```
struct abc {  
    char a;  
    int b;  
    char c;  
};
```

15. What is Structure Packing, and what preprocessor directive is used to enforce it?

16. Discuss the architectural engineering trade-off between choosing structure padding versus structure packing.
17. How do you calculate the total memory allocation size for an array of structures, such as `struct Date dob[4]`; where each `Date` contains three 4-byte integers?
18. Why does the compiler require a terminating semicolon (;) at the very end of a structure definition block?
19. What are the classifications of data types shown in the notes?
Where do structures fit?
20. If you have an uninitialized local structure variable, what values will its members contain?

Answers

1. What is a structure in C, and how does it differ from a standard array?

Answer: A structure is a user-defined, non-primitive data type that groups multiple variables of dissimilar (heterogeneous) data types under a single identifier name. It differs from an array because an array is strictly limited to holding elements that share the exact same data type (homogeneous), while a structure serves as a container for mixed data types.

2. Explain the purpose of the `struct` keyword and what a tag name represents.

Answer: The `struct` keyword is used to establish and define a new custom, user-defined structure type template. The tag name immediately following it acts as a unique identifier or label for that specific structure architecture layout.

3. Are memory locations assigned immediately when a structure template is defined? Explain.

Answer: No. Member variables declared inside a structure template do not receive individual identities or physical memory allocations when the template is defined. Memory is only allocated when an actual structure variable instance is instantiated in the program.

4. Write out the template definition for a structure named `student` containing a character array `name`, an integer `roll number`, and a float `marks` value.

Answer:

```
struct student {  
  
char name[20];  
  
int rollno;  
  
float marks;  
  
};
```

5. What are the two distinct methods available to instantiate structure variables?

Answer:

Method 1 (Separately) : Declaring variables after the template definition, e.g., ``struct Date d1, d2;``

Method 2 (Immediately) : Appending variable names directly between the closing brace and the terminating semicolon of the template layout definition, e.g., ``struct Date { int d, m, y; } d1, d2;``

6. What is a Sequential Initializer List, and what happens if you provide fewer values than the total number of structure members?

Answer:

A sequential initializer list assigns values to a structure variable at the time of declaration using a comma-separated list enclosed in braces (e.g., `{28, 9, 2024}``). The values must match the exact order of the member definitions. If you provide fewer values than the total number of members, the compiler automatically initializes all remaining empty slots to zero (`0``).

7. Which operator is used to access structure members through a direct instance variable?

Answer:

The member access operator, commonly known as the dot operator (``.``), is used to read or modify individual parameters inside a direct structure instance (e.g., ``d1.d = 29;``).

8. Write a code snippet showing how to use ``scanf()`` to read a structure variable with a custom forward slash (``/``) delimiter scheme.

Answer:

```
struct Date d1;  
scanf("%d/%d/%d", &d1.d, &d1.m, &d1.y);
```

9. How can you smoothly copy all properties from one structure variable instance to another variable instance of the exact same structure type?

Answer: You can copy all member properties instantly by writing a direct assignment statement using the equality operator (e.g., `d2 = d1;`), provided both variables are instances of the exact same structure type.

10. Can you pass complete structure instances as parameter variables to a function, or return entire structure blocks from a function?

Answer: Yes. A structure data block can be passed as a complete pass-by-value parameter instance into a function, and it can also be safely returned as a unified data type value from a function definition (e.g., `return somedate;`).

Important Interview Answers

11. Explain the purpose of the arrow operator (->) and write its completely identical bracketed indirection equivalent notation.

Answer: The arrow operator (->) provides a clean syntax alternative to reference and access individual members of a structure through an active object pointer variable. Its completely identical bracketed indirection notation is: **(*ptr).member** (e.g., `p->d` is equivalent to `(*p).d`).

12. What is the role of the typedef keyword when defining structures? Provide an example of how it cleans up object declarations.

Answer: The typedef keyword creates a shorter, user-defined type alias for a long structure layout definition. This allows you to completely omit the mandatory struct keyword during subsequent object declarations.

```
typedef struct student {  
    int rollno;  
} stu;  
stu s; ----> Instead of: struct student s;
```

13. Explain what Structure Padding is and why processors read data in fixed alignments rather than 1 byte at a time.

Answer: Processors read system memory in fixed, continuous chunks called **word sizes** (typically 4 bytes at a time on 32-bit systems or 8 bytes at a time on 64-bit systems) to maximize memory access efficiency. To prevent data from straddling word boundaries—which forces the CPU to waste cycles performing multiple cross-word memory fetches—the compiler automatically inserts empty space bytes (**padding bits**) between adjacent structure members to keep them aligned with word boundaries.

14. Given the following structure, calculate its total evaluated size in bytes on a standard 32-bit CPU boundary, accounting for padding:

```
struct abc {  
    char a;
```

```
int b;  
char c;  
};
```

Answer: The total evaluated size is **12 bytes**. It aligns across three 4-byte words as follows:

char a consumes 1 byte, followed by 3 bytes of internal padding to align the next member to a 4-byte boundary.

int b consumes a full 4-byte word.

char c consumes 1 byte, followed by 3 bytes of trailing padding to ensure the entire structure matches the 4-byte word alignment requirement.

15. What is Structure Packing, and what preprocessor directive is used to enforce it?

Answer: Structure packing compresses a structure by eliminating all internal and trailing padding gaps, forcing the compiler to use byte-level alignment instead. It is explicitly enforced using the preprocessor directive: **#pragma pack(1)**.

16. Discuss the architectural engineering trade-off between choosing structure padding versus structure packing.

Answer: * **Structure Padding** prioritizes **processing speed**. It consumes extra RAM by creating empty padding gaps, but it allows the CPU to fetch data members in a single cycle.

Structure Packing prioritizes **memory conservation**. It saves RAM space by compressing the data layout, but it can degrade overall execution speed because the CPU may require multiple unaligned read cycles to assemble split data blocks.

17. How do you calculate the total memory allocation size for an array of structures, such as struct Date dob[4]; where each Date contains three 4-byte integers?

Answer: An array of structures provisions contiguous memory allocation blocks where each index slot represents a complete structural sequence. The calculation is:

Total Memory = Number of elements * Size of an individual structure instance

For struct Date dob[4], each entry is 12 bytes (3* 4 bytes), making the total size: 4 * 12 bytes} = 48 bytes}.

18. Why does the compiler require a terminating semicolon (;) at the very end of a structure definition block?

Answer: The terminating semicolon indicates the end of the structure statement block. Because C allows you to instantly instantiate structure variables right between the closing brace and the semicolon, the semicolon tells the compiler that the data definition statement is complete and no variables are being declared at that point.

19. What are the classifications of data types shown in the notes? Where do structures fit?

Answer: Data types are split into two major groups:

Primitive (User-Defined / Core Built-in): Basic types such as int, char, float, and double.

Non-Primitive (Derived / Secondary): Grouped, complex tracking interfaces where **Structures** reside.

20. If you have an uninitialized local structure variable, what values will its members contain?

Answer: If declared locally inside a function without initialization, its members are not automatically reset to zero. Instead, they will inherit unpredictable **garbage values** present in that memory location, similar to standard uninitialized local variables.